

File Input/Output and Exceptions

Introduction to Programming

Rae Harbird, r.harbird@ucl.ac.uk

File I/O

More strings

File handling

Exceptions

File I/O

Overview

- ▶ Python programs can take input from files and send output to files.
- ▶ Python can deal with two types of files:
 - ▶ **Text files** - files containing (roughly speaking) a sequence of characters. Text files can be opened in a text editor such as Notepad.
 - ▶ **Binary files** - files containing (roughly speaking) a sequence of binary digits. Binary files are good for storing e.g. images or audio or (structures of) objects.
- ▶ **Note:** We will only cover text files in this lecture because that is all you will need for the coursework.

More strings

Useful string functions

As we are dealing with text files in this lecture, we will be using more of the string handling features in Python.

Every line in a text file is a string and Python has a lot of string processing methods that we can use to process the contents, for example:

Method name	Description
<code>string.strip()</code>	remove the character or characters given as a parameter from both sides of the string, default whitespace
<code>string.rstrip()</code>	remove the character or characters given as a parameter from the right hand end of a string
<code>string.split()</code>	split the string into elements using the character given as a parameter, default is whitespace
<code>string.count()</code>	return the number of occurrences of the string given as a parameter
<code>string.index()</code>	return the start position of the string given as a parameter

Stripping whitespace from strings

- ▶ Take this string, there's a lot of whitespace either side of the text:

```
>>> sentence = '\t \n  This is a test string. \t\t \n\n\n'  
>>> print(sentence)
```

```
    This is a test string.
```

- ▶ Removing leading whitespace

```
>>> print(sentence.lstrip())  
This is a test string.
```

- ▶ Removing trailing whitespace

```
print(sentence.rstrip())
```

```
    This is a test string.
```

Stripping whitespace from strings (continued)

- ▶ Removing both leading and trailing whitespace

```
print(sentence.strip())  
This is a test string.
```

Searching for substrings

- ▶ Count the number of occurrences

```
sentence = "Be nice to yu turkeys dis christmas\nCos' turkeys just
```

```
>>> sentence.count('turkey')
```

```
4
```

- ▶ Find the position of a string from the lefthand side

```
>>> sentence.index('Turkey')
```

```
68
```

- ▶ Search for position in slice

```
>>> len(sentence)
```

```
131
```

```
>>> sentence.count('turkey', 68, 131)
```

```
2
```

```
>>> sentence.count('turkey', 68)
```

```
2
```

Substring in a string?

- ▶ Position

```
sentence.index('cool')  
80
```

- ▶ Determining whether a string contains a substring

```
>>> 'cool' in sentence  
True
```

```
>>> 'stuffing' in sentence  
False
```

```
>>> 'turkey' not in sentence  
False
```

Replacing substrings

```
>>> sentence.replace('turkey', 'chestnut roast')
```

```
"Be nice to yu chestnut roasts dis christmas\nCos' chestnut roasts  
just wanna hav fun\nTurkeys are cool, chestnut roasts are wicked\nAn every chestnut roast has a Mum."
```

Splitting and joining substrings

- ▶ Splitting strings is especially important in file processing. Each line of a file is read in as a string.

```
>>> sentence_list = sentence.split()
>>> sentence_list
```

```
['Be', 'nice', 'to', 'yu', 'turkeys', 'dis', 'christmas',
 'Cos\\', 'turkeys', 'just', 'wanna', 'hav', 'fun', 'Turkeys',
 'are', 'cool,', 'turkeys', 'are', 'wicked', 'An', 'every',
 'turkey', 'has', 'a', 'Mum.']
```

- ▶ There's still a problem with the separator here. We can just remove the offending words or we can use regular expressions. Regular expressions are not covered in this course.
- ▶ Joining the elements of a string is similar:

```
>>> '<--->'.join(sentence_list)
'Be<--->nice<--->to<--->yu<--->turkeys<--->dis<--->
christmas<--->Cos\\'<--->turkeys<--->just<--->wanna
<--->hav<--->fun<--->Turkeys<--->are<--->cool,<--->
turkeys<--->are<--->wicked<--->An<--->every<--->turkey
<--->has<--->a<--->Mum.'
```

Comparison operators (review)

Example	Result
'Orange' == 'orange'	False - Python does not ignore the case of the characters
'Orange' != 'orange'	True
'Orange' < 'orange'	True
'Orange' <= 'orange'	True
'Orange' > 'orange'	False
'Orange' >= 'orange'	False

Character testing methods

We will see that when processing a file, we often need to check that the content is as we expect. We might need to check that, for example, the characters we have read in are digits. Python has several functions that can help with this. Note: the table does not contain a complete list.

Character testing functions	Description
<code>isalnum()</code>	Returns True if the string only contains alphanumeric characters.
<code>isalpha()</code>	Returns True if the string only contains alphabetic characters.
<code>isdigit()</code>	Returns True if the string only contains digits ('0', '2', etc).
<code>isnumeric()</code>	Returns True if the string only contains a number without +, - or a decimal place.
<code>isdigit()</code>	Returns True if the string only contains digits.

File handling

Overview (cont.)

- ▶ Python programs connect to files via objects called streams.
- ▶ Files themselves are represented by objects of class `File` in Python programs.
- ▶ Because they rely on the physical presence and state of actual files and directories, Python programs that use file I/O are prone to unexpected errors
 - ▶ ... and so should include exception handling in their file I/O methods.

Syntax

The name of the file to open

Store the returned
file objects in variables.

```
infile = open("input.txt", "r")
```

```
outfile = open("output.txt", "w")
```

Specify the mode for the file:
"r" for reading (input)
"w" for writing (output)

Read data from `infile`.

Write data to `outfile`.

Close files after the
data is processed.

```
infile.close()
```

```
outfile.close()
```

If you fail to close an output
file, some data may not be
written to the file.

Other methods

A complete list of methods that you can use with files can be found [here] (<https://docs.python.org/3/library/io.html>).

Method name	Description
<code>file.close()</code>	indicates that you are done reading/writing the file
<code>file.flush()</code>	writes any buffered data to an open output file
<code>file.read()</code>	reads and returns the entire file as a string
<code>file.readable()</code>	returns True if the file can be read
<code>file.readline()</code>	reads and returns a single line of the file as a string
<code>file.readlines()</code>	reads and returns the entire file as a list of line strings
<code>file.seek(position)</code>	sets the file's current input cursor position
<code>file.tell()</code>	returns the file's current input cursor position
<code>file.writable()</code>	returns True if the file can be written
<code>file.write("text")</code>	sends text to an output file
<code>file.writelines(lines)</code>	sends a list of lines to an output file

Opening a file

- ▶ A **stream** allows for the flow of data between a program and some I/O device or file. When we open a file using Python we are creating a file object that will allow us to use the file.
- ▶ An **input stream** allows data flow into the program. For example:

```
inFile = open("input.txt", "r")    # inFile is a file object
```

- ▶ An **output stream** allows data flow out of the program. For example:

```
outFile = open("output.txt", "w")  # outFile is a file object too
```

Gotcha

- ▶ If the file doesn't exist, Python will create the file for you. Be careful though, when you open a file for writing, Python immediately opens it and erases all the current contents. If you want to append content to a file then you should use:

```
appendFile = open("output.txt", "a")
```

- ▶ You will get an error if:
 - ▶ the file doesn't exist.
 - ▶ the file does exist but is read-only and you've asked to write to it.
 - ▶ you don't have permission to read or write to the file.
- ▶ We will see how to handle these errors later.

Closing a file

- ▶ After you have finished with a file, you must close it before the program ends. If you don't, the data you have written to the file may not all be saved properly.

```
infile.close()  
outfile.close()
```

Reading line by line

- ▶ Python makes it very easy to read a file line by line.

```
shoppingListFile = open("shopping_list.txt", "r")

line = shoppingListFile.readline()
while line != "":
    line = shoppingListFile.readline().rstrip()
    print(line)

shoppingListFile.close()
```

- ▶ Better still:

```
with open("shopping_list.txt", "r") as file:
    lineCount = 0
    for line in file:
        print("next line:", line.rstrip())
        lineCount += 1
    print("Line count:", lineCount)

file.close()
```

Token based processing

- ▶ Sometimes we need to split lines up and process all the elements in a line one by one. This is known as token based processing.
- ▶ Data is frequently stored as Comma Separated Values (CSV) in text files. For example: the file Trends_in_UK_butterflies.csv.

```
Species of the wider countryside (24),,,,
,Long-term change (1976-2016),Trend,Short-term change (2011-2016),Trend
brimstone,9,No change,105,No change
(Gonepteryx rhamni),,,,
brown argus,-25,No change,13,No change
(Aricia agestis),,,,
comma,138***,Increased,2,No change
(Polygonia c-album),,,,
common blue,-24,No change,1,No change
(Polyommatus icarus),,,,
```

- ▶ Each line in the file is a record of readings pertaining to one species of butterfly and the fields in the record are separated by commas.
- ▶ We can use what we have learned so far to process this file.

Splitting the line

Let's say that we wanted to read every line of our file containing butterfly data and that we want to extract the name of the butterfly, the latin name of the butterfly, the short-term change and the population trend.

.... but we need to write an algorithm first ...

The algorithm

Species of the wider countryside (24),,,,
,Long-term change (1976-2016),Trend,Short-term change (2011-2016),Trend
brimstone,9,No change,105,No change
(Gonepteryx rhamni),,,,
brown argus,-25,No change,13,No change
(Aricia agestis),,,,
comma,138***,Increased,2,No change
(Polytonia c-album),,,,

butterflies.py

```
with open(BUTTERFLY_FILE) as file:
    line_count = 0
    butterflyName = ""
    shortTermChange = ""
    trend = 0

    for line in file:
        if line_count >= HEADER_LINES:
            line = line.rstrip()
            if line_count % 2 == 0:
                butterflyName, *extra, shortTermChange,
                trend = line.split(',')
            else:
                latinName = line.strip(',')
                print("{} {}: \t {}, {}".format(
                    butterflyName.capitalize(),
                    latinName, shortTermChange.rstrip('*'),
                    trend))

            line_count += 1
    print("\nLine count:", line_count)
```

A problem

- ▶ Here is a file containing actor's names:

```
Evan Rachel Wood  
Will Smith  
Oscar Isaac Hernandez Estrada  
James Tiberius Kirk  
Phillip Seymour Hoffman  
Sarah Jessica Parker  
Tommy Lee Jones
```

- ▶ Sometimes lines in a file will not necessarily have the same number of fields.
- ▶ Likewise, we may only need to process some of the fields in a line and not others.
- ▶ This won't work:

```
>>> with open("film_stars.txt") as file:  
...     first, middle, last = file.read().split()
```

```
Traceback (most recent call last):
```

```
  File "<input>", line 2, in <module>
```

```
ValueError: too many values to unpack (expected 3)
```

Using the gather operator '*'

► General syntax:

```
name, name, ..., *name = string.split("delimiter")
```

Table 5: Table : Effect of *extra at different places

# extra at start	Oscar Isaac Hernandez Estrada
*extra, first, last = ...	_-----/
	*extra first last
# extra in middle	Oscar Isaac Hernandez Estrada
first, *extra, last = ...	_-----/
	first *extra last
# extra at end	Oscar Isaac Hernandez Estrada
first, last, *extra = ...	_-----/
	first last *extra

Note: from Building Python Programs, Obourn, Reges, Stepp

Reading numbers

- ▶ You may want to process a file, or parts of it, as numbers. How can we do that?
- ▶ See program: `compute_sum.py`

```
# This program adds numbers from a file and prints the sum.
```

```
def main():  
    sum = 0.0  
    with open("bad_numbers.txt") as file:  
        for n in file.read().split():  
            # try to convert n to a float, but if it  
            # is not a valid float, print error message  
            try:  
                sum += float(n)  
            except ValueError:  
                print("Invalid number:", n)  
    print("Sum is:", round(sum, 1))  
  
main()
```

Writing to a file

- ▶ We can write the string “Hello, World!” to our output file using the statement:

```
outfile.write("Hello, World!\n")
```

- ▶ When writing text to an output file, you must explicitly write the newline character to start a new line. You can also write formatted strings to a file with the write method:

```
outfile.write("Number of entries: %d\nTotal: %8.2f\n"  
             % (count, total))
```

- ▶ You can also use the print() function, as long as you specify where you want the output to go:

```
print("Hello, World!", file=outfile)
```

- ▶ If you don't want a newline, use the end argument:

```
print("Total: ", end="", file=outfile)
```

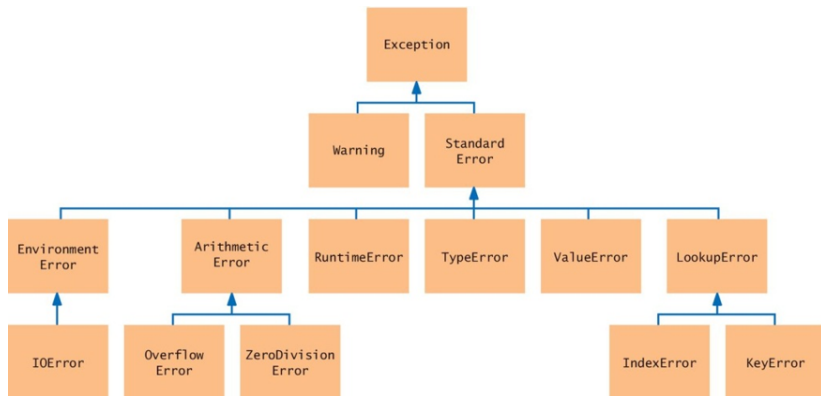
Exceptions

When do exceptions happen?

- ▶ When the Python interpreter encounters a problem that it cannot deal with, it will throw an exception and you will see the errors in the console.

```
>>> int(input("Enter a number:"))
Enter a number:a
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'a'
>>>
```

Python exceptions



From: Python for Everyone, Horstmann and Necaie

Examples

► ZeroDivisionError

```
num_prizes = int(input("Enter the number of prizes: "))
num_students = int(input("Enter the number of students: "))
prizes_per_student = num_prizes / num_students
print("Number of prizes per student: ", prizes_per_student)
```

► Output:

Traceback (most recent call last):

File "C:/Users/Rae/.PyCharmEdu2019.2/config/scratches/scratch_13"
line 3, in <module>

prizes_per_student = num_prizes / num_students

ZeroDivisionError: division by zero

Examples (continued)

► IndexError

```
lis = ["cat", "dog", "hamster", "dromedary"]
```

```
for i in range(1, len(lis) + 1):  
    print(lis[i])
```

► Output:

```
dog
```

```
Traceback (most recent call last):
```

```
File "C:/Users/Rae/.PyCharmEdu2019.2/config/scratches/scratch_14  
line 4, in <module>
```

```
hamster
```

```
    print(lis[i])
```

```
IndexError: list index out of range
```

```
dromedary
```

Examples (continued)

► KeyError

```
dict = {"cat": "meeow!", "dog": "woof!", "hamster": "eek!",  
        "dromedary": "hurrumph!"}
```

```
dict.pop("pig")
```

► Output:

```
Traceback (most recent call last):
```

```
File "C:/Users/Rae/.PyCharmEdu2019.2/config/scratches/scratch_15  
line 2, in <module>
```

```
    dict.pop("pig")
```

```
KeyError: 'pig'
```

Catching an exception

- ▶ Surround the code that might fail with a `try ... catch` block. Here is an example showing how your code could work using the python REPL:

```
>>> try :  
...     int(input("Enter a number: "))  
... except ValueError :  
...     print("That is not a number.")  
...  
Enter a number: g  
That is not a number.  
>>>
```

Catching an exception (cont.)

- ▶ In a program, your code could look like this:

```
## Checks whether the string passed as a parameter is an integer.  
#  
# @return True or False.  
#  
def isInteger(number) :  
    try:  
        int(number)  
        return True  
    except ValueError:  
        return False
```

- ▶ The point here is that you don't have to terminate the program, you can choose what action to take if a value is invalid.

Catching more than one exception

You should write code that allows for code to catch multiple exceptions. Look at `src/analyzedata.py`

```
done = False
while not done :
    try :
        filename = input("Please enter the file name: ")
        data = readFile(filename)

        # As an example for processing the data, we compute the sum.
        ...

    except IOError :
        print("Error: file not found.")

    except ValueError :
        print("Error: file contents invalid.")

    except RuntimeError as error :
        print("Error:", str(error))
```

Code that will generate an exception

Following on from the previous page, the code that will generate and exception looks like this:

```
def readData(inFile) :  
    line = inFile.readline()  
    numberOfValues = int(line) # May raise a ValueError exception.  
    data = []  
  
    for i in range(numberOfValues) :  
        line = inFile.readline()  
        value = int(line) # May raise a ValueError exception.  
        data.append(value)  
  
    # Make sure there are no more values in the file.  
    line = inFile.readline()  
    if line != "" :  
        raise RuntimeError("End of file expected.")  
  
    return data
```

... and finally

- ▶ There may be some actions that you want to perform whether your code generated an exception or not. This is the purpose of the finally clause in a try block:

```
## Opens a file and reads a data set.  
# @param filename the name of the file holding the data  
# @return a list containing the data in the file  
#  
def readFile(filename) :  
    inFile = open(filename, "r")  
    try :  
        return readData(inFile)  
    finally :  
        inFile.close()
```

- ▶ In this case we **always, always** want to close a file before ending the program.

Raising an exception

- ▶ You can raise your own exceptions. You might want to do this when your program encounters a value which fails your validation tests:
- ▶ In `analyzedata.py`, an exception is raised when there is unaccounted for data at the end of the file:

```
# Make sure there are no more values in the file.  
line = inFile.readline()  
if line != "" :  
    raise RuntimeError("End of file expected.")  
  
if bankBalance - withdrawal < 0:  
    raise ValueError("Sorry, you cannot go overdrawn on  
                    this account")
```